PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Secure Process Scheduler with Malicious Job Detection and Persistent Database Queue Scheduling Job database

Team Name: *Team #*	*JobSentinel OSDBMS-V-2026-T172*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Kala Shivansh–2026123* *24021777@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Khan Aabish–2025001* *240211520@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Ghildiyal Anchita–2025073* *240221675@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Negi Priyanshu–2026092* *24011078@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

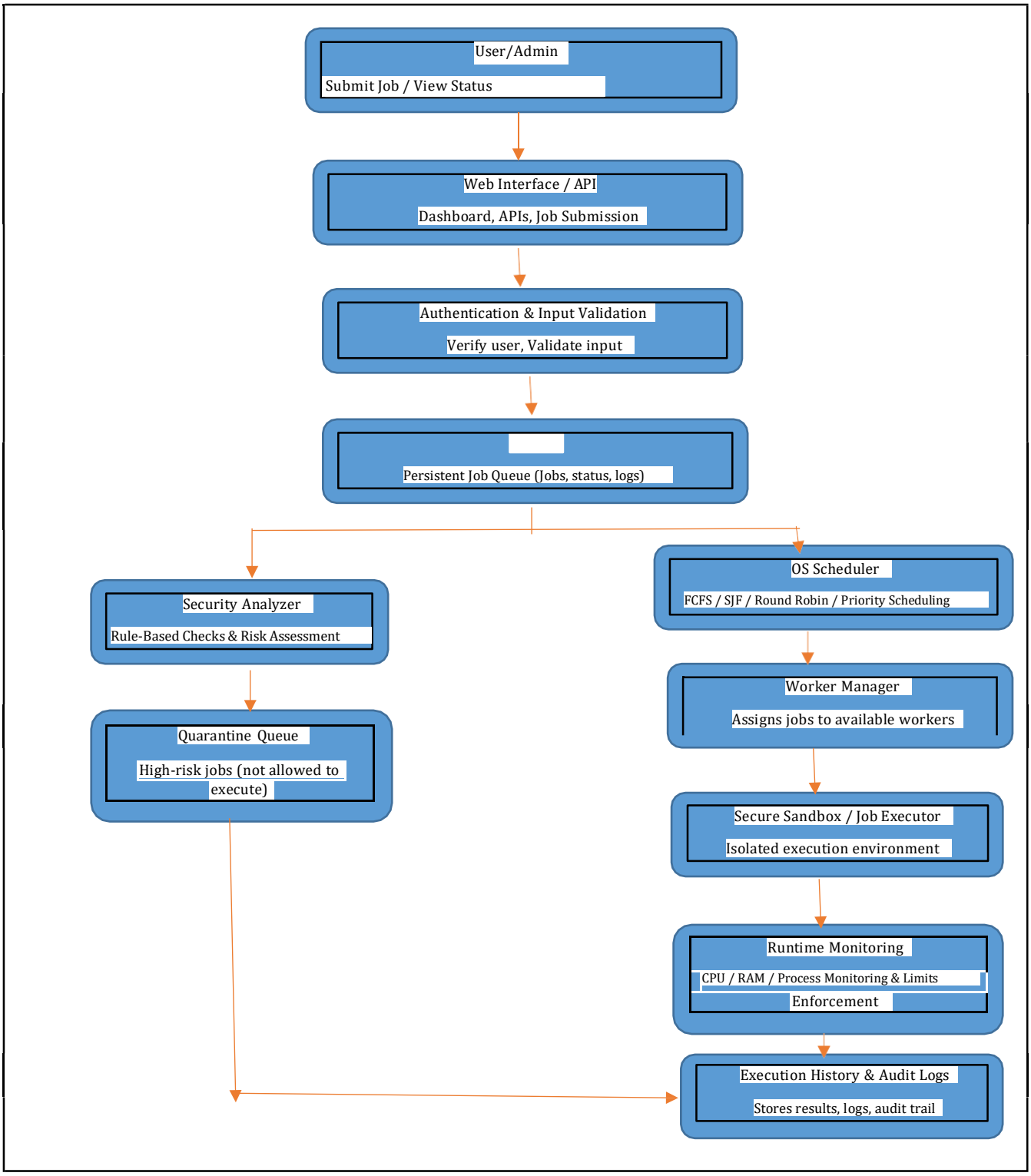
Our goal with JobSentinel is simple: build a process scheduler that is both secure and reliable, pulling together ideas from Operating Systems, DBMS, and cybersecurity. A typical scheduler just orders processes by things like arrival time, burst time, or priority, and that works fine on its own, but it leaves two big gaps. If something crashes, job details can disappear, and nothing actually stops a suspicious job from running in the first place. To close those gaps, we store job information in a MySQL database so the details and execution records stick around. The system supports four algorithms: FCFS, SJF, Round Robin, and Priority Scheduling. Users can submit a job, pick an algorithm, and then follow along with its status and results.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

At this stage we have gone with MySQL instead of PostgreSQL. The backend will be written in Python using FastAPI, and the database will hold job status, job details, and job history. Jobs move through the system like this: Job submission → MySQL database → Algorithm selection → Scheduler → Worker → Job execution → Results and history. Each job carries a command, arrival time, burst time, and priority, and a freshly submitted job starts out as "pending." From there the scheduler pulls jobs out of the database and orders them using FCFS, SJF, Round Robin, or Priority scheduling. When a job is picked up to run, its status flips to "worker" and execution begins. The backend keeps the status current and stores the results and execution history. Later on, we could build a dashboard so users can submit jobs and check on status and history themselves. To keep the work manageable, we have split the project into four parts: the backend and MySQL, the scheduling algorithms, the worker and process execution, and the frontend. Each of us can build our own piece as long as we stick to a shared format and interface. The cybersecurity side — malicious-job detection and quarantine — is set aside for the next phase. That part will bring in sandboxing for untrusted jobs, quarantine for anything malicious, runtime monitoring, and recovery.

SYSTEM ARCHITECTURE



Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
• MySQL database integration • Job submission and management • FCFS, SJF, Round Robin, and Priority Scheduling • Worker assignment and job execution • Execution results and history management	• Priyanshu Negi • Aabish Khan • Anchita Ghildiyal • Shivansh Kala

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

One thing that keeps tripping us up is keeping job information in MySQL accurate when a status changes partway through scheduling or execution, and also figuring out how to handle jobs that fail and where to keep their results. Another challenge is making sure all four algorithms produce the correct execution order and that we can compare their performance in a way that actually means something. Our first proposal listed a bunch of optional extras — malicious-job detection, sandboxing, monitoring, crash recovery — but trying to build all of that at once would have eaten far too much time, so we pushed those to later phases instead.

Tasks Pending

(Describe the main tasks that remain to be completed. Max 250 words).

Task Pending	Team Member
• Malicious-job detection • Security risk assessment • Quarantine management • Secure sandbox execution • Runtime monitoring and recovery mechanisms	• Priyanshu Negi • Anchita Ghildiyal • Aabish Khan • Shivansh Kala

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

What we are aiming for is a working demo of JobSentinel where a user can submit a job, save it to MySQL, choose an algorithm, and watch it run. The system should keep track of job status and execution details so that finished and failed jobs can be looked back at. Being able to compare how FCFS, SJF, Round Robin, and Priority Scheduling treat the same sample jobs is a nice bonus. In later versions, we expect the system to detect, quarantine, launch, monitor, and recover from malicious jobs.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

Right now we are at the planning and architecture stage of the current workflow. We have pinned down the main components, settled on MySQL, and worked out how the modules should talk to each other. Our immediate focus is getting the database and backend interfaces built and tested while the rest of the team works on the scheduler, the worker, and the frontend. Once those are ready, we will integrate everything and run it together as a full demo. We cannot give an exact completion percentage yet, because the source-code ZIP has not been available for inspection. The next milestone is a demo of the whole flow, from submitting a job all the

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

JobSentinel is written in Python with FastAPI, and MySQL handles the database. The code is organised into files and folders based on what they do, such as database files, job submission, the scheduling algorithms, workers, and the frontend. Repository link: https://github.com/Shivansh8007/Secure-Process-Scheduler-with-Malicious-Job-Detection

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	**Status (Pass/Fail)**	**Notes**
1) Core System Validation 2) End-to-End Workflow Testing	Pass Pass	The core job scheduling is validated. We ran the whole path — job submission, MySQL storage, scheduling, worker execution, and result/history — and it passed

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Done for Phase 2: the base system, which covers MySQL, job management, FCFS, SJF, Round Robin, Priority Scheduling, worker execution, and execution history. Still to go for Phase 3: the cybersecurity side, where malicious jobs get detected, risk-assessed, quarantined, and run securely where needed